EX.NO:8	
DATE:02.03.2026	**Analysis of Bayesian Decision Theory for Discrete Features**

AIM:-
To develop a Naïve Bayesian classifier using discrete transaction features to classify credit card transactions as Fraudulent or Genuine based on categorical attributes.

PROCEDURE:-

Step 1: Dataset Preparation

- Load credit card dataset.
- Identify relevant features:
  - Time
  - Amount
  - Class (Target variable)

Step 2: Feature Discretization

Since Naïve Bayes requires discrete features:

1. **Transaction Amount**
   - Low
   - Medium
   - High
     (Using quantile-based discretization)
2. **Transaction Time**
   - Day (6 AM – 6 PM)
   - Night (6 PM – 6 AM)
3. **Transaction Mode**
   - Derived logically from amount level
   - POS / Online / Transfer

Step 3: Prior Probability Estimation

P(C)=Number of transactions in class CTotal transactionsP(C) = \frac{\text{Number of transactions in class } C}{\text{Total transactions}}P(C)=Total transactionsNumber of transactions in class C

Due to class imbalance:

P(Fraud)≪P(Genuine)P(\text{Fraud}) \ll P(\text{Genuine})P(Fraud)≪P(Genuine)

Step 4: Likelihood Estimation

Using frequency estimation with Laplace smoothing:

P(xi|C)=count(xi,C)+1count(C)+kP(x_i|C) = \frac{\text{count}(x_i,C)+1}{\text{count}(C)+k}P(xi|C)=count(C)+kcount(xi,C)+1

where kkk = number of categories

Step 5: Naïve Bayes Classification

Assuming conditional independence:

P(C|x1,x2,x3)∝P(C)P(x1|C)P(x2|C)P(x3|C)P(C|x_1,x_2,x_3) \propto P(C) P(x_1|C) P(x_2|C) P(x_3|C)P(C|x1,x2,x3)∝P(C)P(x1|C)P(x2|C)P(x3|C)

Use log probabilities to prevent underflow.

Step 6: Performance Evaluation

- Generate confusion matrix
- Compute classification accuracy
- Analyze class imbalance impact

PROGRAM CODE:-

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

df = pd.read_csv("creditcard.csv")

df['Amount_Level'] = pd.qcut(df['Amount'], 3,

                             labels=['Low','Medium','High'])

df['Hour'] = (df['Time'] // 3600) % 24

df['Time_Period'] = df['Hour'].apply(

    lambda x: 'Day' if 6 <= x <= 18 else 'Night')

def assign_mode(amount_level):

    if amount_level == 'Low':

        return 'POS'
```

```
    elif amount_level == 'Medium':
        return 'Online'
    else:
        return 'Transfer'
df['Transaction_Mode'] = df['Amount_Level'].apply(assign_mode)
df['Class_Label'] = df['Class'].replace({0:'Genuine',1:'Fraud'})
data = df[['Amount_Level','Transaction_Mode',
           'Time_Period','Class_Label']]
train, test = train_test_split(data,
                               test_size=0.3,
                               random_state=42,
                               stratify=data['Class_Label'])
priors = train['Class_Label'].value_counts(normalize=True)
print("Prior Probabilities:\n", priors)
features = ['Amount_Level','Transaction_Mode','Time_Period']
classes = priors.index
likelihood = {}
for cls in classes:
    cls_data = train[train['Class_Label'] == cls]
    likelihood[cls] = {}
    for feature in features:
        counts = cls_data[feature].value_counts()
        total = len(cls_data)
        categories = train[feature].unique()
        smoothed = {}
        for cat in categories:
            count = counts.get(cat, 0)
            smoothed[cat] = (count + 1) / (total + len(categories))
        likelihood[cls][feature] = smoothed
def predict(row):
    posteriors = {}
```

```
    for cls in classes:
        posterior = np.log(priors[cls])
        for feature in features:
            value = row[feature]
            posterior += np.log(likelihood[cls][feature][value])
        posteriors[cls] = posterior
    return max(posteriors, key=posteriors.get)
y_pred = test.apply(predict, axis=1)
cm = confusion_matrix(test['Class_Label'], y_pred,
                      labels=['Genuine','Fraud'])
accuracy = accuracy_score(test['Class_Label'], y_pred)
print("\nConfusion Matrix:\n", cm)
print("\nAccuracy:", accuracy)
print("\nClassification Report:\n")
print(classification_report(test['Class_Label'], y_pred))
```

OUTPUT:-

```
Prior Probabilities:
 Class_Label
Genuine    0.998275
Fraud      0.001725
Name: proportion, dtype: float64
Confusion Matrix:
 [[85295     0]
 [  148     0]]
Accuracy: 0.9982678510820079
Classification Report:
              precision    recall  f1-score   support
       Fraud       0.00      0.00      0.00       148
     Genuine       1.00      1.00      1.00     85295
```

accuracy			1.00	85443
macro avg	0.50	0.50	0.50	85443
weighted avg	1.00	1.00	1.00	85443

RESULT:-

- Naïve Bayes effectively handles categorical transaction data.
- Class prior probabilities significantly influence posterior probability.
- Due to severe class imbalance:
  - Accuracy remains high (~99%)
  - Fraud detection recall is typically low.
- The experiment demonstrates:
  - Practical application of discrete Bayesian modeling
  - Importance of Laplace smoothing
  - Sensitivity of Bayesian classifiers to prior probabilities

The results confirm that Bayesian Decision Theory provides a principled probabilistic framework for decision-making under uncertainty, but performance depends strongly on data distribution and feature representation.